

S32K3XX PIT & STM - RTD

Automotive Systems & Applications Engineering Team
GPIS-BL, Automotive Processing



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.





AGENDA

PIT

1. [PIT Features Overview](#)
2. [PIT Configuration](#)
3. [PIT API Functions and Example Code](#)

STM

1. [STM Features Overview](#)
2. [STM Configuration](#)
3. [STM API Functions and Example Code](#)

GPT PIT Overview



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



PERIODIC INTERRUPT TIMER (PIT) OVERVIEW

- The PIT is an array of timers that can be used to raise interrupts and triggers.
- Includes a dedicated **Real Time Interrupt (RTI)**, which runs on a separate clock and can be used for system wakeup.

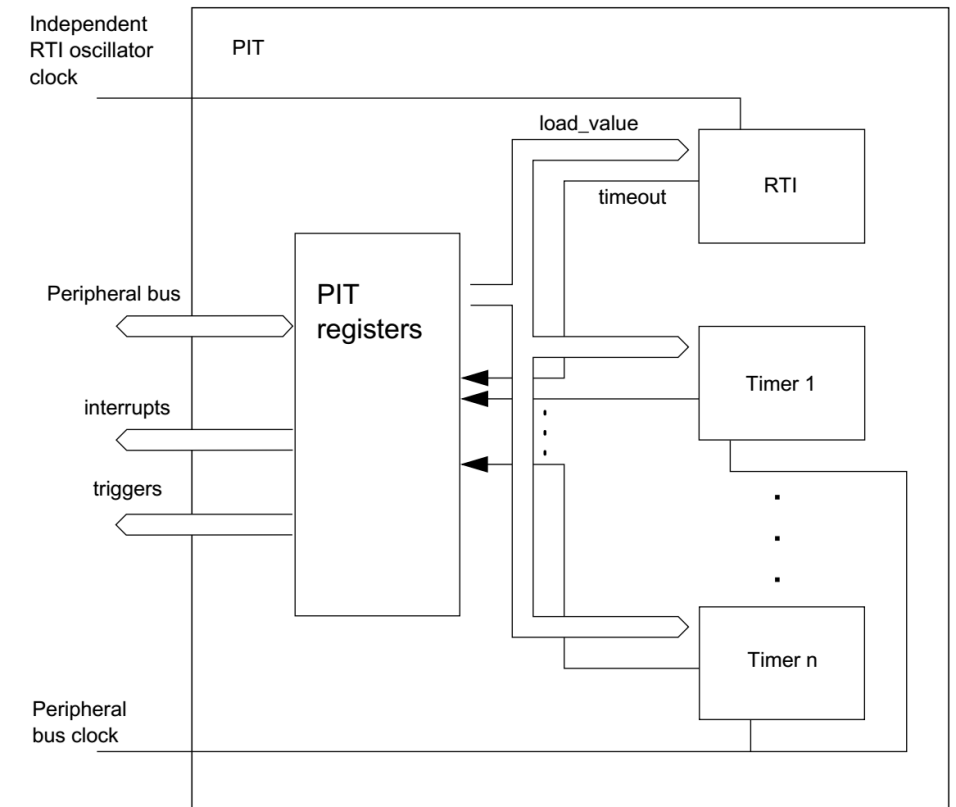
Instance	K3x4	K342	K312	K311
PIT_0	Yes	Yes	Yes	Yes
PIT_1	Yes	Yes	Yes	Yes
PIT_2	Yes	Yes	No	No

- The S32K3xx has **3** instances of the PIT:

Feature	PIT_0	PIT_1	PIT_2
Real Time Interrupt (RTI)	Yes (32-bit)	No	No
Lifetime Timer	Yes	No	No
Chaining to create a 64-bit Timer	Yes	No	No
Number of 32-bit channels	4	4	4

PERIODIC INTERRUPT TIMER (PIT) FEATURES OVERVIEW

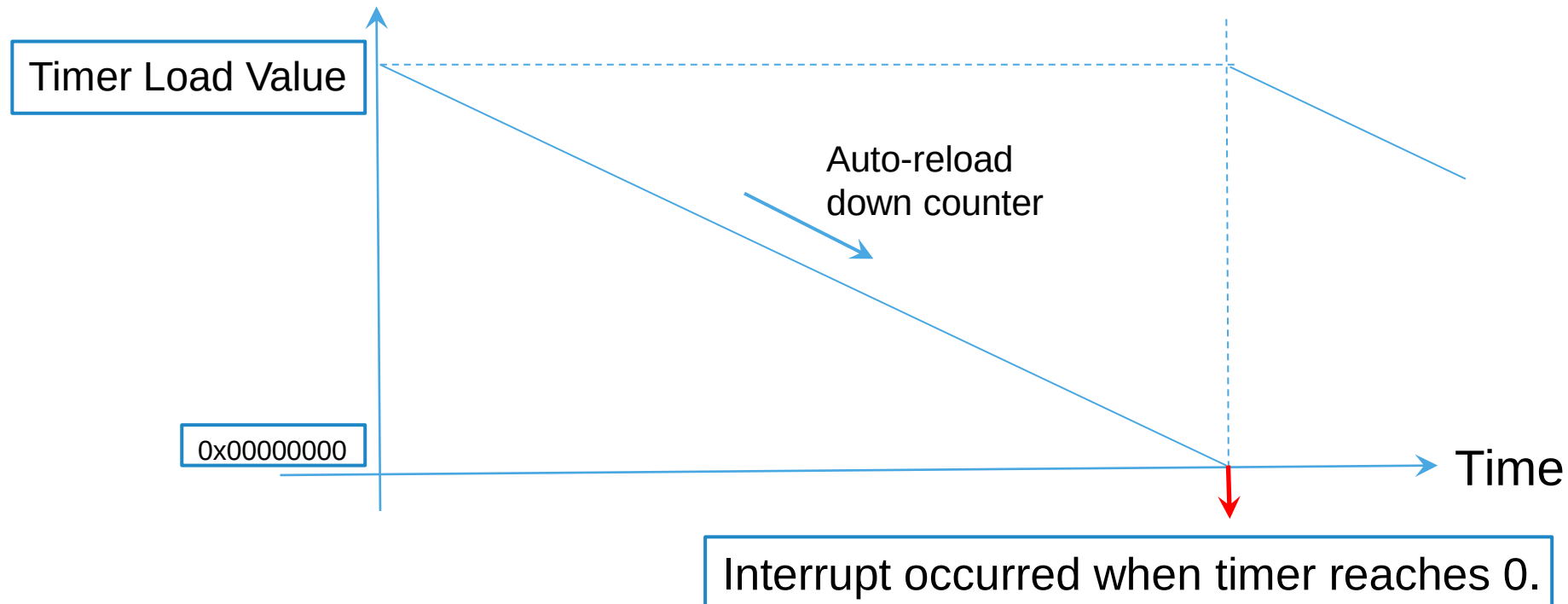
- The key features of the module are:
 - One RTI timer to wakeup the CPU
 - Ability of timers to generate trigger pulses
 - Ability of timers to generate interrupts
 - Maskable interrupts
 - Option to raise RTI interrupt, even when the bus clock is switched off
 - Power saving with a separate input clock for the RTI timer. All other timers share a common core clock.
 - Independent timeout periods for each timer
 - All the timers use a down-counter
- When a timer has chain mode enabled, it counts after the previous timer has expired. If timer n-1 counts down to 0, counter n decrements the value by one.
- The first timer (Timer 0) cannot be chained to any other timer.



Block diagram of PIT_RTI

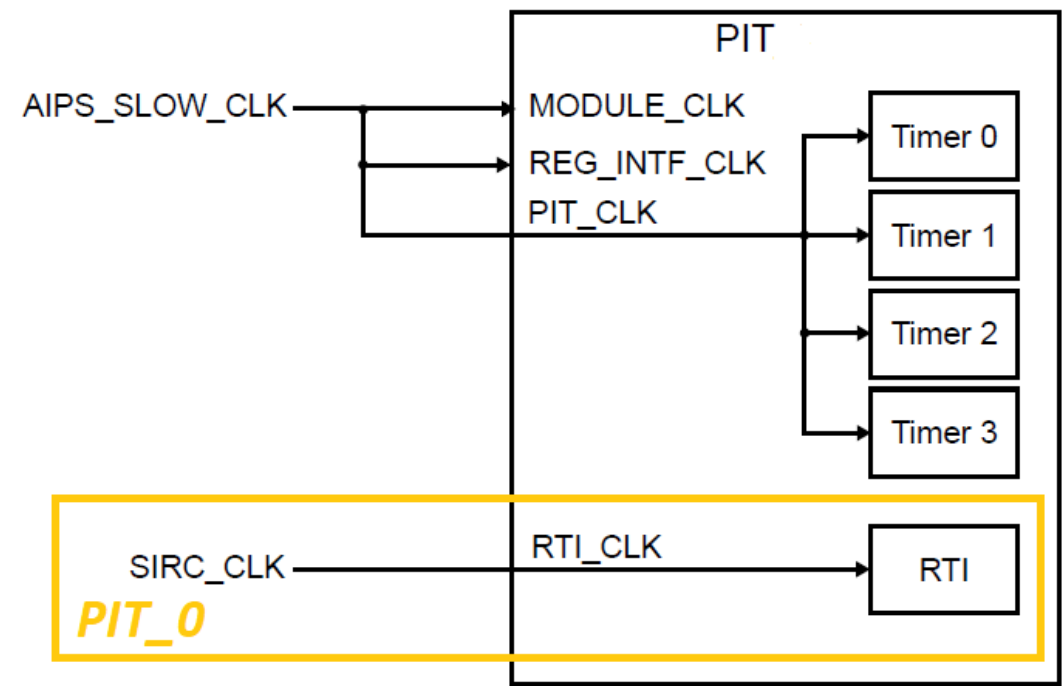
PERIODIC INTERRUPT TIMER (PIT) FEATURES OVERVIEW

- Four 32-bit timer channels with clock source from AIPS_SLOW_CLK
- One RTI timer in PIT_0 runs on SIRC (32 KHz) and supports operation in STANDBY mode.
- Ability of timers to generate trigger pulses and interrupts
- Independent timeout periods for each timer
- All the timers use a down-counter
- PIT_0 timers can be chained to create 64bit timer



PERIODIC INTERRUPT TIMER (PIT) CLOCK AND INTERRUPT

- Timers and modules are clocked by “AIPS_SLOW_CLK”
- All the **PIT** instances can generate periodic triggers that can be routed via **TRGMUX** to motor control IPs such as **eMIOS, LCU, BCTU, ADC**.
- The real time interrupt (PIT_0_RTI) is clocked by the 32 KHz **SIRC**.



- All PIT channels interrupts including the RTI share an IRQ interrupt vector

Address	Vector (all cores)	IRQ	NVIC IPR register number	NVIC non-IPR register number	INTM support	Instance	Interrupt request	S32K314	S32K324	S32K344
0x000001C0	112	96	24	3	Y	PIT_0	Interrupt for Channel0	YES	YES	YES
0x000001C4	113	97	24	3	Y	PIT_1	Interrupt for Channel0	YES	YES	YES
0x000001C8	114	98	24	3	Y	PIT_2	Interrupt for Channel0	YES	YES	YES

GPT PIT Configuration



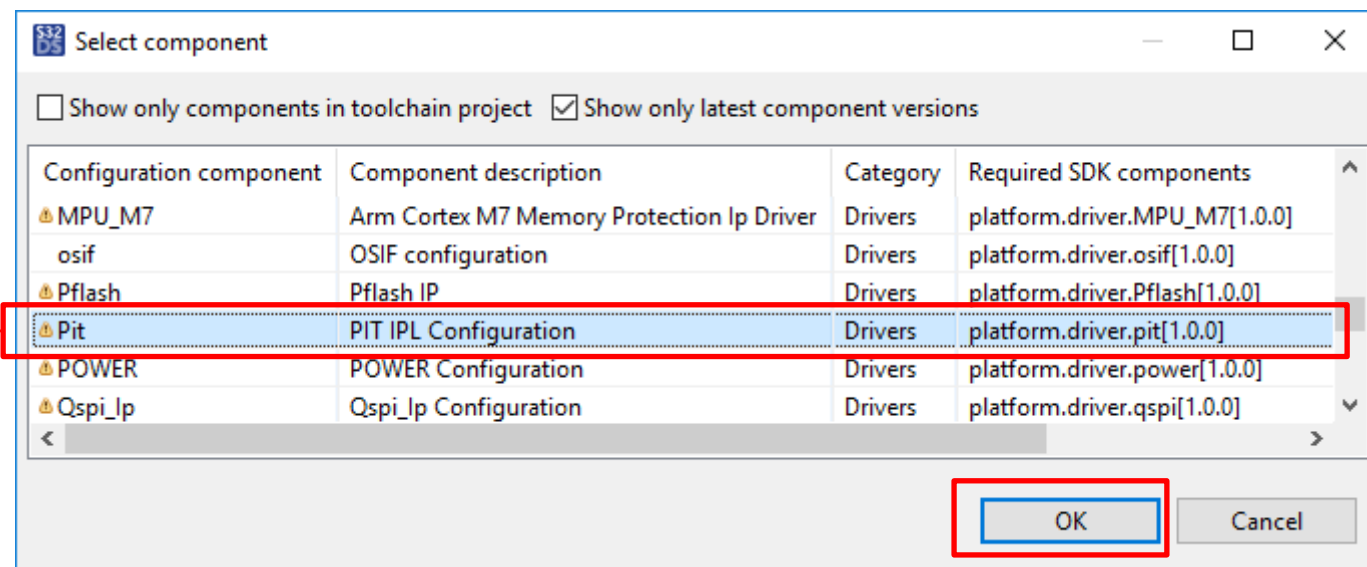
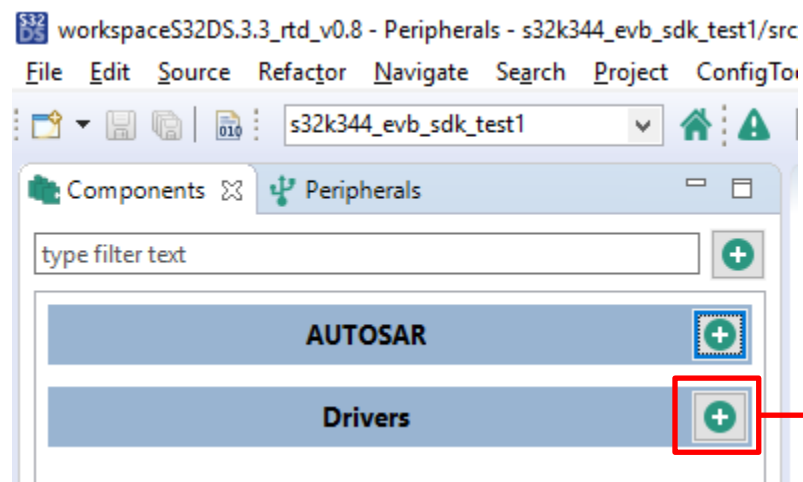
SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



ADD PIT COMPONENT INTO THE PROJECT



PIT CONFIGURATION

The screenshot shows the 'PIT IPL Configuration [Drivers]' window. It has a 'Name' field with 'Pit_1' and a 'Mode' dropdown set to 'PIT Mode'. Below this is a 'PIT' section with a tabbed interface. The 'GptChannelConfigSet' tab is selected and highlighted with a red box and a callout '1'. Inside this tab, there is a 'GptPit' section with a '+' button highlighted by a red box and callout '2'. Below 'GptPit', there is a list of PIT instances. The first instance, 'Pit_0', is selected and highlighted with a red box and callout '3'. Within the 'Pit_0' configuration, the 'Pit Freeze Enable' checkbox is checked and highlighted with a red box and callout '4'. Below this is the 'GptPitChannels' section. The first channel, 'PitChannel_0', is selected and highlighted with a red box and callout '5'. Within this channel configuration, the 'Pit Channel' dropdown is set to 'CH_0' and highlighted with a red box and callout '6'. The 'ChainMode' checkbox is unchecked and highlighted with a red box and callout '7'. The 'Pit Notification' dropdown is set to 'Pit0_Ch0_Callback' and highlighted with a red box and callout '7'.

1. Select the tab “GptChannelConfigSet”
2. Click “plus” button to add a timer channel if necessary
3. Select the PIT_0 ~ PIT_2 instance to configure
4. Check “PitFreezeEnable” if the timer should be paused in debug mode
5. Select the timer channel within the PIT
6. “ChainMode” can be enabled if 64-bit timer is needed.
7. Set the callback function which is called when timeout occurred on this channel.

- PIT timer clock source is PLAT_SLOW_CLK (usually 40 MHz) without clock prescaler.
- PIT RTI clock source is SIRC (32 KHz).

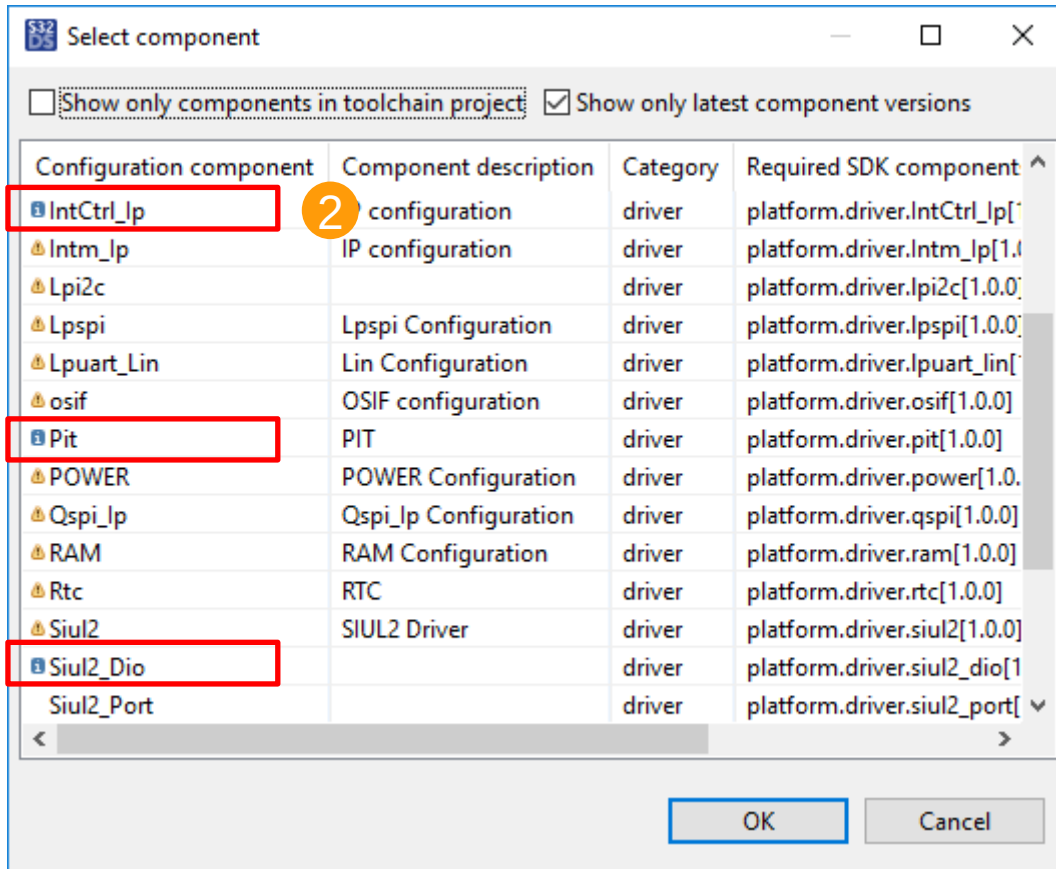
PIT CONFIGURATION

^ PIT				
GptChannelConfigSet GptHwConfiguration GptAutosarExt GptDriverConfiguration				
#	Name	GPT Peripheral ISR Name	GptIsrEnable	GptChannellsUsed
0	GptHwConfiguration_0	PIT_0_CH_RTl	☐	☐
1	GptHwConfiguration_1	PIT_0_CH_0	☒	☒
2	GptHwConfiguration_2	PIT_0_CH_1	☐	☐
3	GptHwConfiguration_3	PIT_0_CH_2	☐	☐
4	GptHwConfiguration_4	PIT_0_CH_3	☐	☐
5	GptHwConfiguration_5	PIT_1_CH_0	☐	☐
6	GptHwConfiguration_6	PIT_1_CH_1	☐	☐
7	GptHwConfiguration_7	PIT_1_CH_2	☐	☐
8	GptHwConfiguration_8	PIT_1_CH_3	☐	☐
9	GptHwConfiguration_9	PIT_2_CH_0	☐	☐
10	GptHwConfiguration_10	PIT_2_CH_1	☐	☐
11	GptHwConfiguration_11	PIT_2_CH_2	☐	☐
12	GptHwConfiguration_12	PIT_2_CH_3	☐	☐

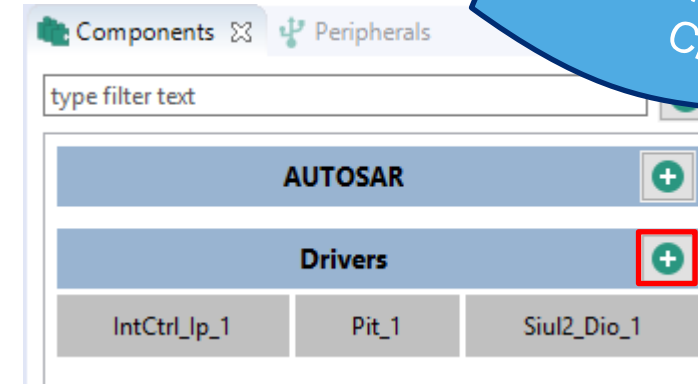
Enable the ISR for the selected PIT timer channels if this timer interrupt is needed.

ADD INTERRUPT CONTROL (INTCTRL) FOR PIT EXAMPLE

- Select IntCtrl_Ip to add interrupt controller which is needed for PIT interrupts
- Select Siul2_Dio for LED toggling test purpose



2 Add IntCtrl_Ip, Pit, Siul2_Dio into the project.



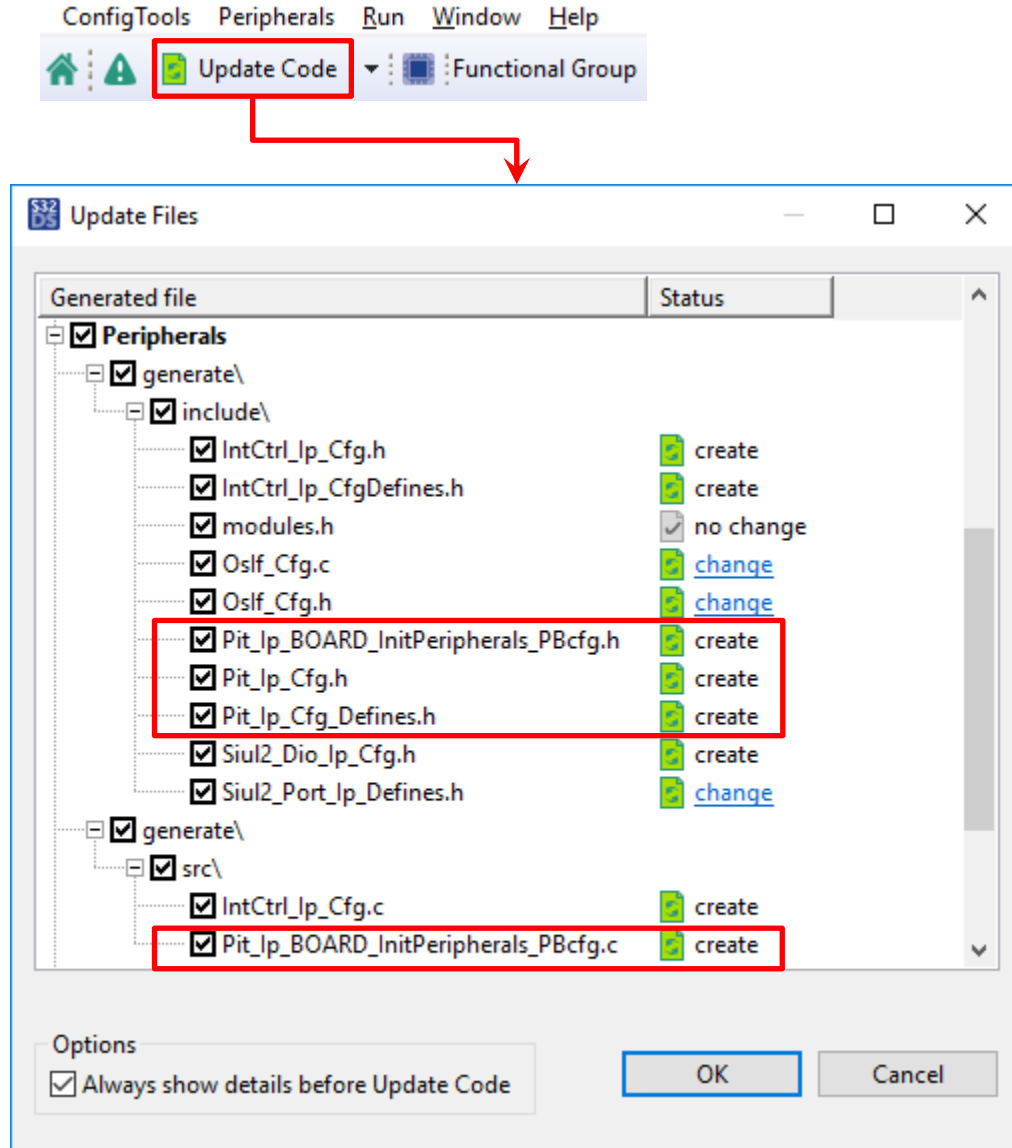
To see the LEDs configuration. Check the S32K3 RTD Training Pins and Clocks

Source	Platform	LCU	LCU IRQn	IntCtrl_Ip_1	Pit_1
75	Platf...	LCU0_IRQn	✓	✓	undefi...
76	Platf...	LCU1_IRQn	✓	✓	undefi...
77	Platf...	PIT0_IRQn	✓	✓	undefi...
78	Platf...	PIT1_IRQn	✓	✓	undefi...
79	Platf...	PIT2_IRQn	✓	✓	undefi...
80	Platf...	RTC_IRQn	✓	✓	undefi...

3

3 Enable PIT0 IRQ in IntCtrl_Ip configuration.

GENERATE SDK CODE AND ADD APPLICATION CODE



- The following files are generated by configuration tool:
 - Pit_Ip_BOARD_InitPeripherals_PBcfg.h
 - Pit_Ip_Cfg.h
 - Pit_Cfg_Defines.h
 - Pit_Ip_BOARD_InitPeripherals_PBcfg.c
- The following driver source files are added into the project:
 - RTD/src/Pit_Ip.c
 - RTD/include/Pit_Ip.h
 - RTD/include/Pit_Ip_Types.h

GPT PIT API Functions



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



PIT API FUNCTIONS

“Pit_Ip.h” must be included in your source code.

Available API Functions can be found in “Pit_Ip.h”. Some are explained below:

```
void Pit_Ip_Init(uint8 instance, const Pit_Ip_InstanceConfigType *config);
```

Initialize a PIT instance with given configuration.

```
void Pit_Ip_InitChannel(uint8 instance, const Pit_Ip_ChannelConfigType *chnlConfig);
```

Initialize a PIT channel with given configuration.

```
void Pit_Ip_EnableChannelInterrupt(uint8 instance, uint8 channel);
```

Enable a PIT channel's interrupt.

```
Pit_Ip_StatusType Pit_Ip_StartChannel(uint8 instance, uint8 channel, uint32 countValue);
```

Start a PIT channel with given counter value.

```
uint64 Pit_Ip_GetCurrentTimer(uint8 instance, uint8 channel);
```

Get the current timer value of a PIT channel.

PIT EXAMPLE CODE

```
/* Including necessary configuration files. */  
#include "Mcal.h"  
#include "Siul2_Port_Ip.h"  
#include "Siul2_Dio_Ip.h"  
#include "Siul2_Icu_Ip.h"  
#include "Clock_Ip.h"  
#include "IntCtrl_Ip.h"  
#include "Pit_Ip.h"  
#include "OsIf.h"  
  
/* Macro definitions */  
#define PIT0_INST    0  
#define PIT0_CH0     0
```

Include necessary header files in main.c

Add macro definitions for PIT instance number and PIT channel number

PIT EXAMPLE CODE

```
/* External functions and variables */  
extern ISR(PIT_0_ISR);
```

```
/* Callback functions definition */
```

```
void Pit0_Ch0_Callback (uint8 channel)
```

```
{
```

```
    (void)channel;
```

```
    Siul2_Dio_Ip_TogglePins(LED_GREEN_PORT, (1<<LED_GREEN_PIN));
```

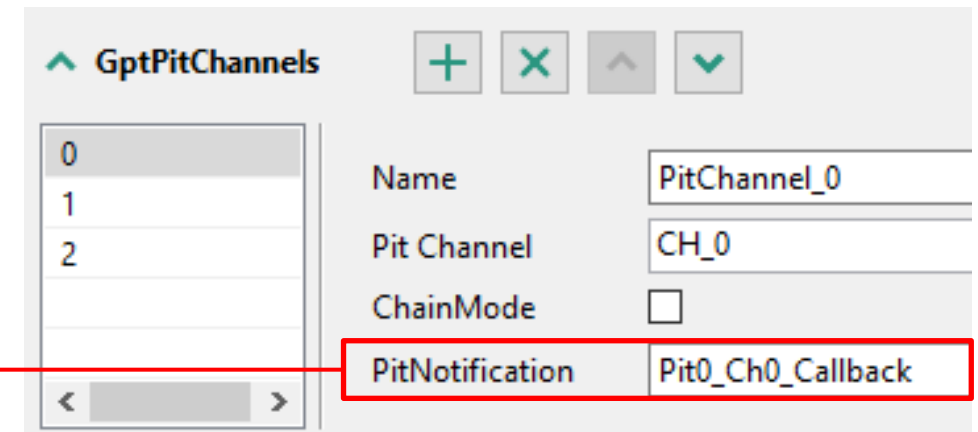
```
    return;
```

```
}
```

The PIT_0_ISR is used for interrupt handler installation.

The callback function is defined for each PIT timer channel.

NOTE: To avoid errors, write the same name for the callback in the PIT controller and the application.



PIT EXAMPLE CODE

```
int main(void)
```

```
{  
    /* Clock initialization */  
    Clock_Ip_Init(&Mcu_aClockConfigPB[0]);
```

Initialize clock settings.

```
    /* SIUL2 pin configuration */  
    Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0, g_pin_mux_InitConfigArr0);  
  
    /* OsIf initialization */  
    OsIf_Init(NULL);
```

Initialize pins settings.

PIT EXAMPLE CODE

```
/* PIT timer and interrupt configuration */
Pit_Ip_Init(PIT0_INST, &PIT_0_InitConfig_PB);
Pit_Ip_InitChannel(PIT0_INST, &PIT_0_ChannelConfig_PB[0]);

/* PIT0_CH0 period is 100mS since its clock is AIPS_SLOW_CLK 40MHz */
Pit_Ip_StartChannel(PIT0_INST, PIT0_CH0, 4000000);
Pit_Ip_EnableChannelInterrupt(PIT0_INST, PIT0_CH0);

/* Install IRQ handler for PIT0 */
IntCtrl_Ip_InstallHandler(PIT0_IRQn, PIT_0_ISR, NULL_PTR);
IntCtrl_Ip_EnableIrq(PIT0_IRQn);
```

Initialize PIT instance

Initialize PIT channels

Start PIT channel 0

Install PIT ISR

Build the application.



Debug the application.



Run the application.



Observe the **GREEN LED** toggling every 100mS on the board.

GPT STM Overview



SECURE CONNECTIONS
FOR A SMARTER WORLD

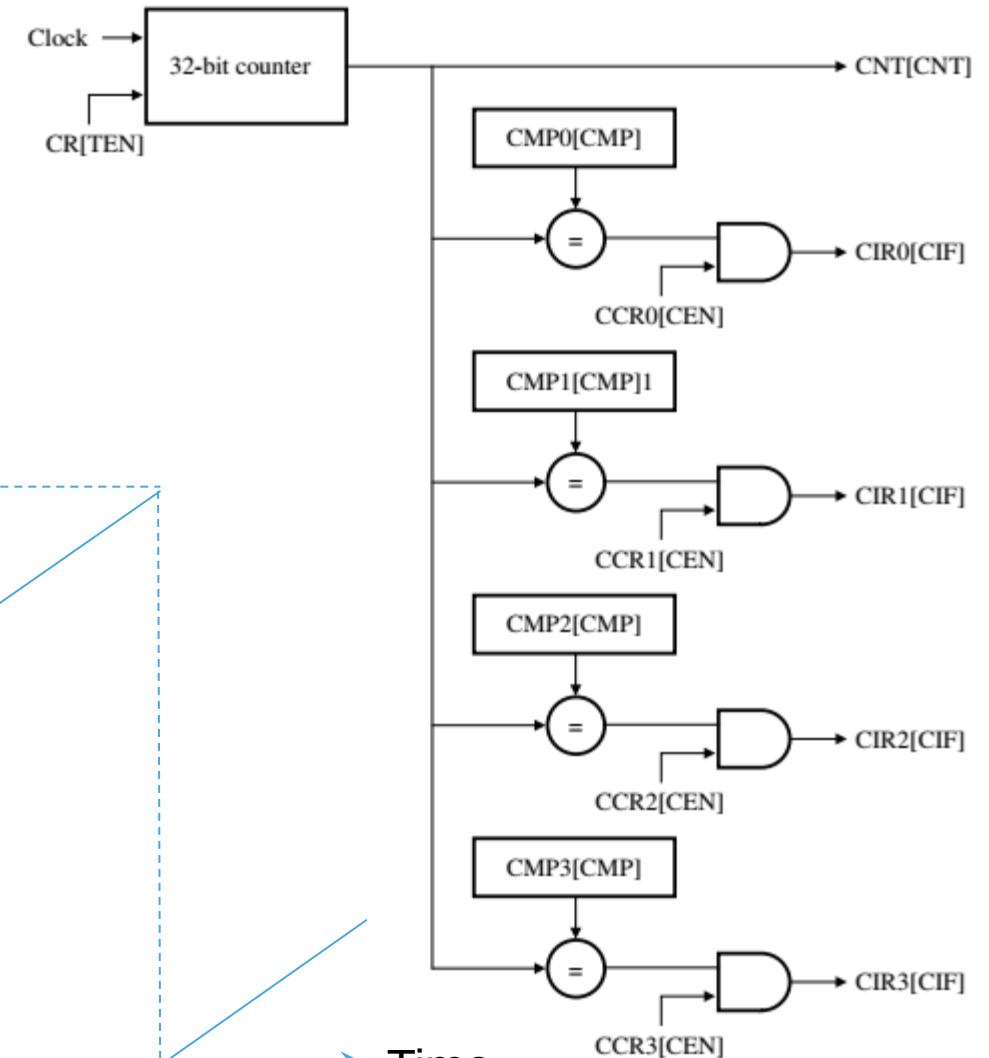
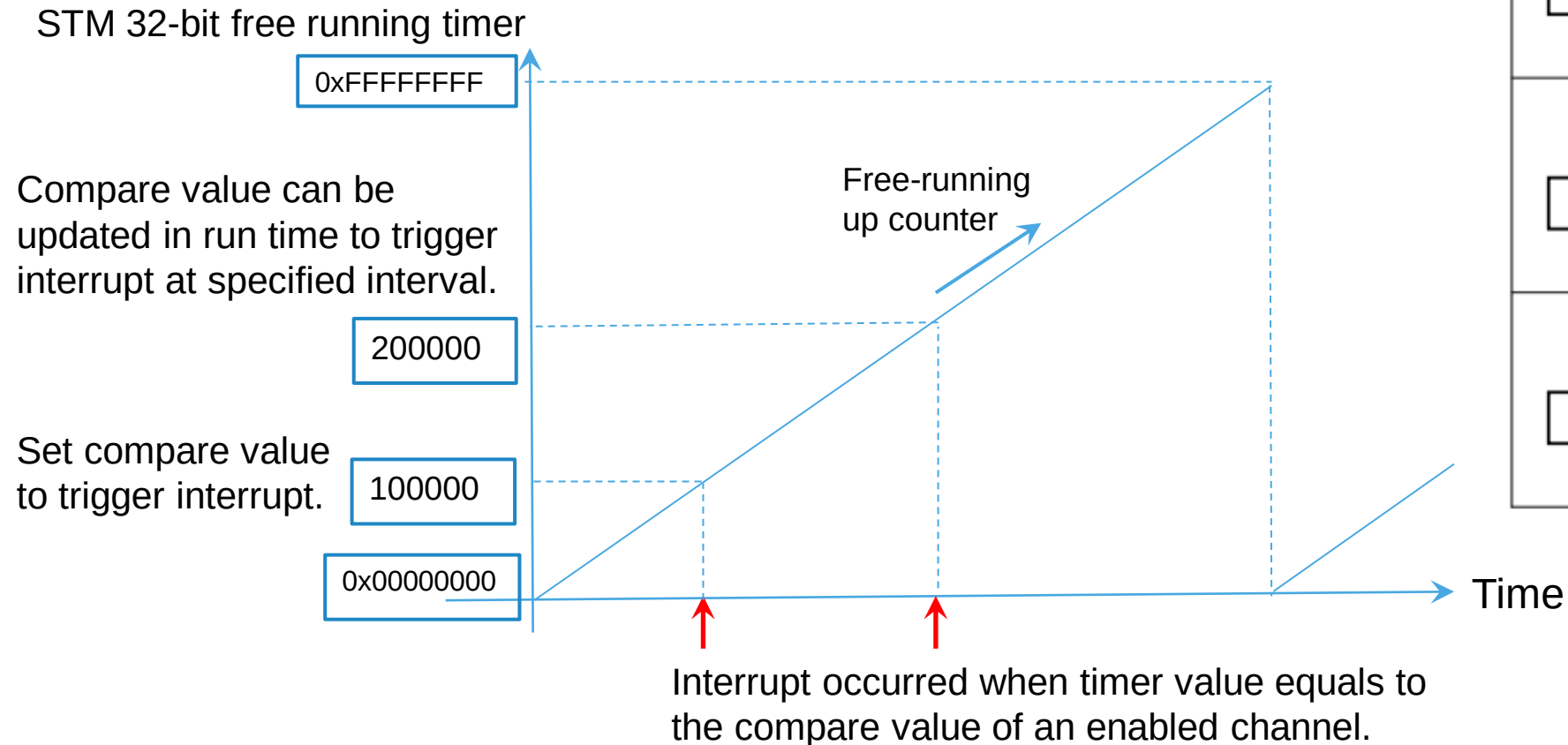
PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



SYSTEM TIMER MODULE (STM) OVERVIEW

- Clock source is selectable in CGM
 - AIPS_PLAT_CLK or FXOSC or FIRC
- One 32-bit up counter with 1~256 prescaler
- Four 32-bit compare channels
- Interrupt source for each compare channel



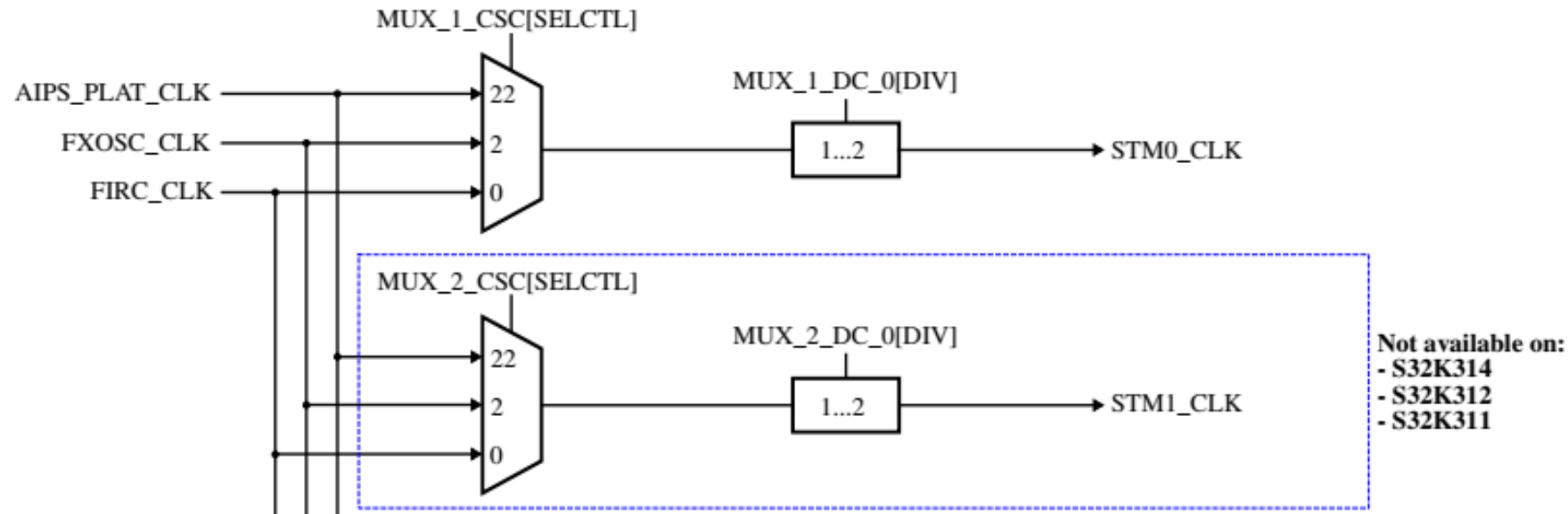
SYSTEM TIMER MODULE (STM) FEATURES OVERVIEW

- When enabled, the counter increments continuously at the module frequency divided by an 8-bit prescaler value.
- STM has four identical compare channels. When a channel is enabled and its compare value matches the timer count, STM sets the channel interrupt flag and generates an IRQ on that channel.

Channel Configuration	Register
Set the channel compare value	CMP [CMP]
Clear channel flag	CIR [CIF]
Enable the compare channel	CCR [CEN]
<i>Repeat for each desired channel</i>	

Timer Configuration	Register
Set the initial timer count.	CNT [CNT]
Configure behavior in Debug mode.	CR [FRZ]
Set the counter prescaler.	CR [CPS]
Start the timer.	CR [TEN]

SYSTEM TIMER MODULE (STM) CLOCK



- STM0 clock comes from CGM clock mux 1.
- STM1 clock comes from CGM clock mux 2.

GPT STM Configuration



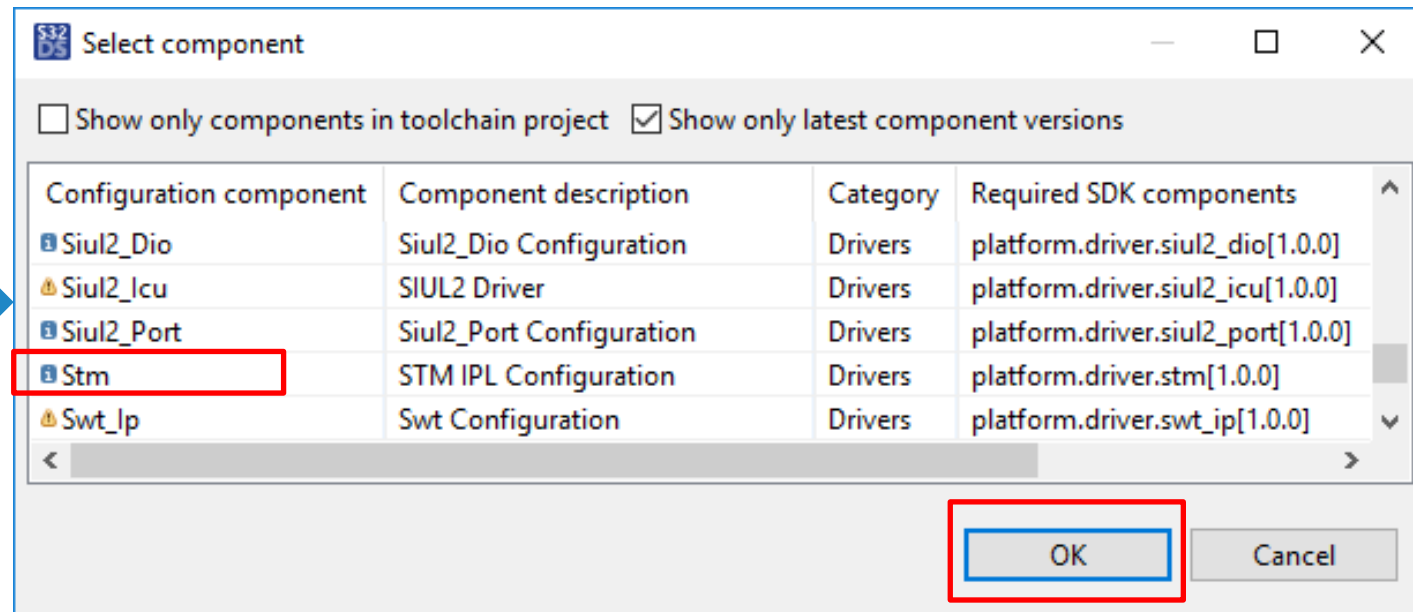
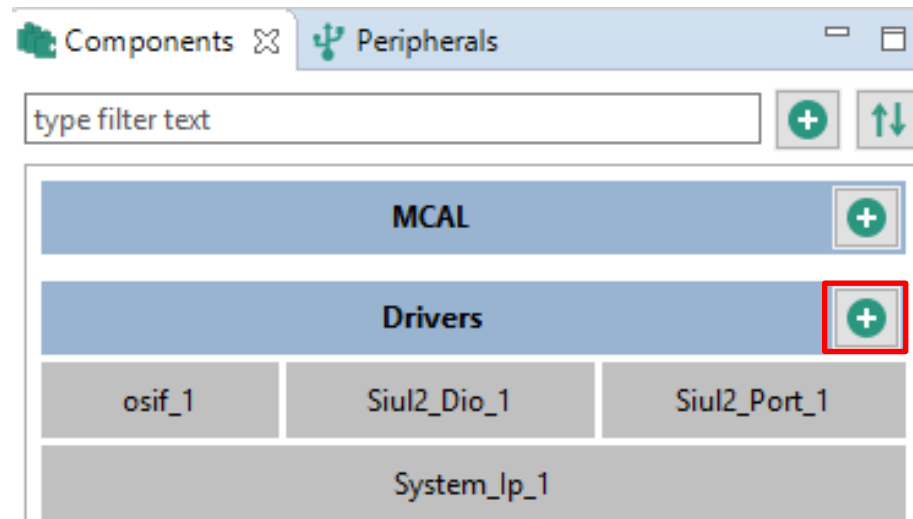
SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



ADD STM COMPONENT INTO THE PROJECT



STM CONFIGURATION

The screenshot shows the STM configuration interface with the following elements and callouts:

- 1**: Points to the **GptChannelConfigSet** tab.
- 2**: Points to the **GptStm** section header.
- 3**: Points to the **Stm Hardware Module** dropdown menu.
- 4**: Points to the **GptStmPrescaler** input field.
- 5**: Points to the **StmFreezeEnable** checkbox.
- 6**: Points to the **GptStmChannels** section header.
- 7**: Points to the **Stm Channel** dropdown menu.
- 8**: Points to the **StmNotification** dropdown menu.

The configuration details shown are:

- Name: Stm0
- Stm Hardware Module: STM_0
- GptStmPrescaler: 48
- GptStmAlternatePrescaler: 48
- StmFreezeEnable: ☒
- GptStmChannels:
 - Name: StmChannel_0
 - Stm Channel: CH_0
 - StmNotification: Stm0_Ch0_Callback

1. Select the tab “GptChannelConfigSet”.
2. Add an STM instance.
3. Select the STM instance – STM0 or STM1.
4. Set the prescaler (1~256).
The alternate prescaler is optional and it's for dual-clock mode.
5. Enable or disable freeze mode.
(Whether pause the timer in debug mode)
6. Add an STM compare channel.
7. Select an STM channel in the STM instance.
8. Set the notification function which is called when timer compare event occurred.

STM CONFIGURATION

STM IPL Configuration *[Drivers]*

Name

Stm_1

Mode

STM Mode

^ STM

GptChannelConfigSet

GptHwConfiguration

GptAutosarExt

GptDriverConfiguration

#	Name	GPT Peripheral ISR Name	GptIsrEnable	GptChannelsUsed
0	GptHwConfiguration_0	STM_0_CH_0	☒	☒
1	GptHwConfiguration_1	STM_0_CH_1	☐	☐
2	GptHwConfiguration_2	STM_0_CH_2	☐	☐
3	GptHwConfiguration_3	STM_0_CH_3	☐	☐
4	GptHwConfiguration_4	STM_1_CH_0	☐	☐
5	GptHwConfiguration_5	STM_1_CH_1	☐	☐
6	GptHwConfiguration_6	STM_1_CH_2	☐	☐
7	GptHwConfiguration_7	STM_1_CH_3	☐	☐

Check the checkbox if the selected STM channel's interrupt is needed.

STM CONFIGURATION

STM

GptChannelConfigSet GptHwConfiguration **GptAutosarExt** GptDriverConfiguration

Name GptAutosarExt

GPT Enable DualClock mode Support	☐	1
GPT Change Next Timeout Value Support	☒	2
GPT Enable User Mode Support	☐	
GPT Chain Mode Support	☐	
GPT Standby Wakeup Support	☐	

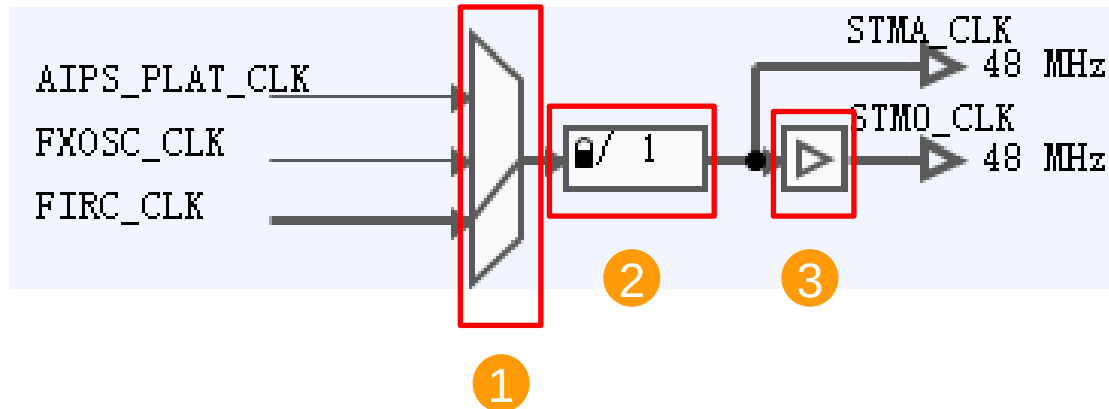
1 Dual Clock mode is used if there are two sets of clock settings in the application. The prescaler and the alternate prescaler are used for the two clock modes, respectively.

- 2
- Enable this option so that the driver code updates the STM channel compare value during every time compare interrupt occurred.
 - Disable this option, then the STM compare interrupt occurred only once for a counter cycle. If repeated interrupt is needed, the interrupt callback function must update the compare value accordingly.

STM CLOCK SOURCE CONFIGURATION



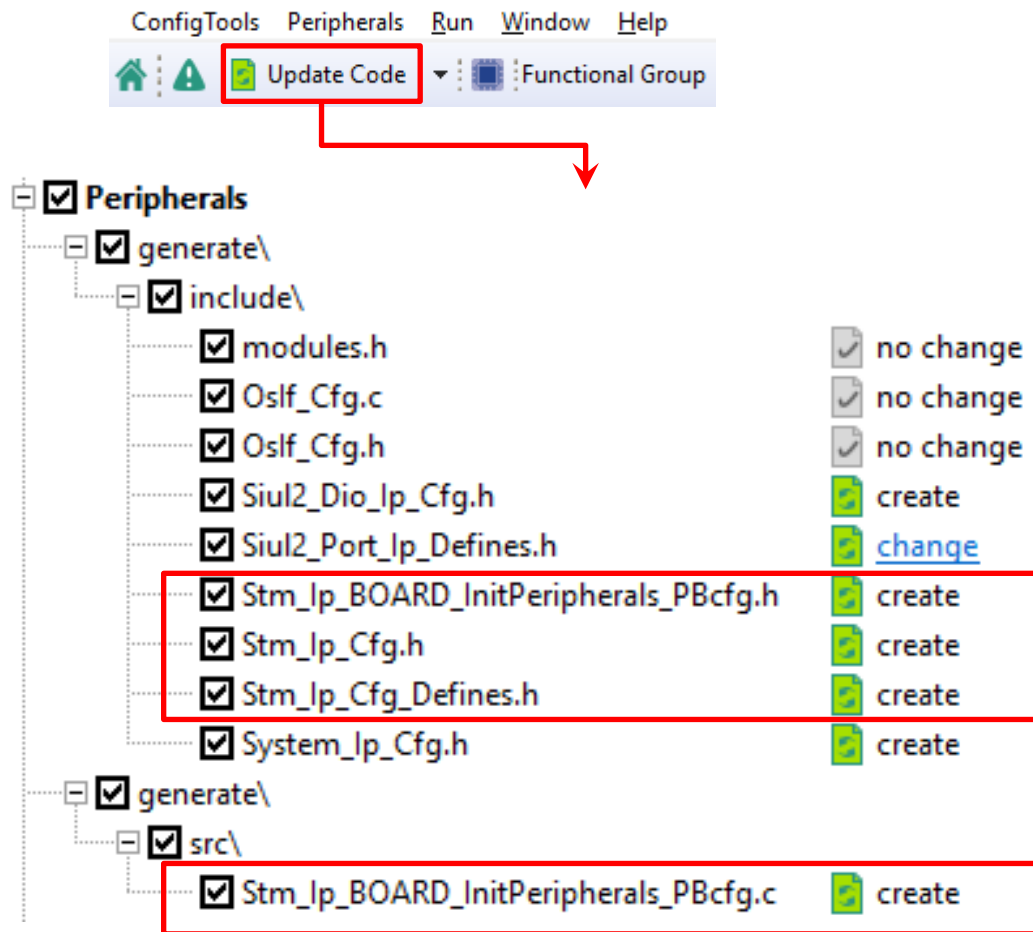
Switch to Clocks Diagram



Configure STM clock:

1. FIRC_CLK as clock source
2. Divider = 1
3. Clock to STM0 enabled

GENERATE SDK CODE AND ADD APPLICATION CODE



- The following files are generated by configuration tool:
 - Stm_Ip_BOARD_InitPeripherals_PBcfg.h
 - Stm_Ip_Cfg.h
 - Stm_Cfg_Defines.h
 - Stm_Ip_BOARD_InitPeripherals_PBcfg.c
- The following driver source files are added into the project:
 - RTD/src/Stm_Ip.c
 - RTD/include/Stm_Ip.h
 - RTD/include/Stm_Ip_Types.h

GPT STM API Functions



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



STM API FUNCTIONS

“**Stm_Ip.h**” must be included in your source code.

Available API Functions can be found in “Stm_Ip.h”. Some are explained below:

```
void Stm_Ip_Init(uint8 instance, const Stm_Ip_InstanceConfigType *configPtr);
```

Initialize the STM instance with given configuration

```
void Stm_Ip_InitChannel(uint8 instance, const Stm_Ip_ChannelConfigType *configPtr);
```

Initialize the STM channel with given configuration

```
void Stm_Ip_StartCounting(uint8 instance, uint8 channel, uint32 compareValue);
```

Reads the current counter register value and sets the compare register to the sum of counter register value plus the timeout value. And enable the compare channel.

```
void Stm_Ip_StartTimer(uint8 instance, uint32 startValue);
```

Set a new start counter value and enables the STM counter

```
void Stm_Ip_EnableChannel(uint8 instance, uint8 channel);
```

Enable the channel selected

```
uint32 Stm_Ip_GetCounterValue(uint8 instance);
```

Get the counter value

STM EXAMPLE CODE

```
/* Including necessary configuration files. */  
#include "Mcal.h"  
#include "Siul2_Port_Ip.h"  
#include "Siul2_Dio_Ip.h"  
#include "Siul2_Icu_Ip.h"  
#include "Clock_Ip.h"  
#include "Intctrl_Ip.h"  
#include "Stm_Ip.h"  
#include "OsIf.h"  
  
/* Macro definitions */  
#define STM0_INST    0  
#define STM0_CH0     0
```

Include necessary header files in main.c

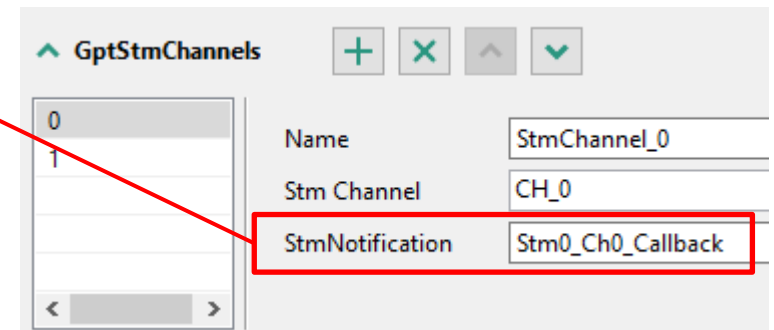
Add macro definitions for STM instance number and STM channel number

STM EXAMPLE CODE

```
/* External Variables and Functions */  
extern ISR(STM_0_ISR);
```

```
/* Callback function or Notification Function */  
void Stm0_Ch0_Callback (uint8 channel)  
{  
    (void)channel;  
    Siul2_Dio_Ip_TogglePins(LED_RED_PORT, (1<<LED_RED_PIN));  
    return;  
}
```

STM notification function should match the function name set in configuration tool.



NOTE: To avoid errors, write the same name for the callback in the STM controller and the application.

STM EXAMPLE CODE

```
int main(void)
{
    /* Clock initialization */
    Clock_Ip_Init(&Mcu_aClockConfigPB[0]);

    /* SIUL2 pin configuration */
    Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0, g_pin_mux_InitConfigArr0);

    /* OsIf initialization */
    OsIf_Init(NULL);

    /* STM configuration, STM clock is determined by CGM settings - default is FIRC 48MHz */
    /* STM timer is enabled here */
    /* STM prescaler=48, so 1cnt=1uS */
    Stm_Ip_Init(STM0_INST, &STM_0_InitConfig_PB);
}
```

Initialize clock settings.

Initialize pins settings.

Initialize STM instance
Timer starts to run.

STM EXAMPLE CODE

```
/* Initialize STM0 compare channel 0 */
Stm_Ip_InitChannel(STM0_INST, &STM_0_ChannelConfig_PB[0]);

/* Start STM0 channel 0 - set compare value, enable this channel */
Stm_Ip_StartCounting(STM0_INST, STM0_CH0, 100000);

/* Install IRQ handler for STM0 */
IntCtrl_Ip_InstallHandler(STM0_IRQn, STM_0_ISR, NULL_PTR);

/* Enable STM0 interrupt in NVIC */
IntCtrl_Ip_EnableIrq(STM0_IRQn);
```

Initialize an STM compare channel.
But the channel is still disabled.

The STM channel is enabled with
given compare value/interval.

Install STM ISR into the vector
table.

Enable STM interrupt in NVIC

Build the application.



Debug the application.



Run the application.



Observe the **RED LED** toggling every 100mS on the board.



S32K3 MICROCONTROLLERS FOR AUTOMOTIVE AND INDUSTRIAL

Tackling cost and complexity of automotive software development



[S32K MCU family page](#)
[S32K3 MCU page](#)



[S32DS for S32 platform page](#)



Online engineering support:

- [S32K MCUs community](#)
- [S32DS community](#)



SECURE CONNECTIONS
FOR A SMARTER WORLD



[SHOWROOM.NXP.COM](https://www.showroom.nxp.com)